- Timescale hierarchy
- Task
    - Timescale unit
    - Cost
        - Continuous discretization
    - Children
        - Parents
        - Deactivating orphans
    - Real task duration
    - Start/end constraints
    - Real task completion time
    - Prerequisites
- Non-overflow constraint
- Decision variables
- Objective function
- Deadlines
- Default cost
- Quantizing Events
- External factors
    - Early biasing
    - Cognitive performance
    - Task cognitive sensitivity
    - Context-switching
    - Encoding context-switching cost

Timescale hierarchy

$U \subseteq \mathbb{N}$ is a set of timescale units.

$\forall a \in U \forall b \in U [a \pmod b \equiv 0]$

$\Theta : U \to \text{Set } \mathbb{N}$

$\Theta(u) = \{x \in \mathbb{N} | x \pmod u \equiv 0\}$

$\Theta(u)$ is the set of all valid starting/ending times and durations for $u \in U$.

$\Upsilon = \min_{u \in U} u$ shall be considered the "atomic" unit.

Task

T is the set of all tasks.

Timescale unit

$u_t \in U$ is the timescale unit of task $t \in T$.

Cost

I_t is the set of cost configurations for task $t \in T$.

$\forall t \in T(I_t \neq \emptyset)$

Each task must have at least 1 cost configuration, otherwise decision variables and cost computation would be undefined.

$$
\begin{aligned}
C_{it} \subseteq \{([a,b],c) | \\
& a \in \theta_t(i) \\
& \wedge b \in \theta_t(i) \cup \{\max[\theta_t(i)] + u_t\} \\
& \wedge b > a \\
& \wedge c \in \mathbb{Z}\}
\end{aligned}
$$

Gives the cost c which applies if the #Real task completion time of task t under cost configuration i falls inside closed interval $[a,b]$.

In other words, no cost configurations should overlap with each other.

Continuous discretization

Suppose we are given a PDF of variable $f(x)$, which represents the risk of non-completion for a given duration allocated to the task.

Let's suppose the absolute cost of non-completion is Q.

Let:

$F(x) = \int_0^x f(x)dx$

The expected cost for a given duration allocation δ is:

$$E[\delta] = Q[1 - F(\delta)]$$

We can then choose a finite set of number of values for δ, and call it Δ.

For any $\delta \in \Delta$ and deadline d, our cost intervals will be:

$$C_{it} = \{([0, d), E[\delta]), ([d, \infty), Q)\}$$

Children

Each task can have multiple sets of children according to cost configuration.

$Ch_{it} \in T$ is the set of children for a task $t \in T$ and cost configuration $i \in I_t$. (should not contain cycles)

However, we also want to be able to decide task cost configurations independently of each other.

Thus, there should never occur a situation where two child configurations are selected, and the same child appears in both.

Therefore, each child must have only one possible parent.

Parents

$$Pa_t = \begin{cases} \iota p(t \in Ch_{D_p[i]p}), & \exists! p \in T(t \in Ch_{D_p[i]p}) \\ \emptyset, & \neg\exists p \in T(t \in Ch_{D_p[i]p}) \end{cases}$$

Pa_t gives the current parent or null of the given task $t \in T$.

$Hc : T \times T \to \text{Predicate}$

$Hc(p, c) = \exists i \in I_p(c \in Ch_{ip})$

$Hc(p, c)$ checks if $p \in T$ has $c \in T$ as a possible child.

$$\forall c \in T \neg(\exists a \in T \exists b \in T[a \neq b \wedge Hc(a, c) \wedge Hc(b, c)])$$

There are no children which have two different possible parents.

$$\forall t \in T(u_t \neq \max U \to Pa_t = \emptyset)$$

There are no tasks whose timescales are not the max timescale with no parents.

> **Note**
>
> If it is unclear what the parent of a task should be (and it is not of the

> max timescale), a "temporary parent" should be created to house it, this way the durations of the children are properly factored into the higher timescales.

Deactivating orphans

A cost configuration $i \in I_t$ is active for a $t \in T$ if:

$$D_t[i] = i$$

A task $t \in T$ is orphaned (deactivated) if:

1. It is not a root task.

$$P_{NR}(t) := Pa_t \neq \emptyset$$

2. No parent cost configuration containing t is active.

$$P_{NA}(t) := p = Pa_t \rightarrow \neg \exists i_p \in I_p \left[t \in Ch_{i_p p} \wedge D_p[i] = i_p \right]$$

If a task is orphaned (deactivated), its real duration must be zero (indicating that the task has been dropped/ignored).

$$P_{NR}(t) \wedge P_{NA}(t) \rightarrow \delta_{it} = 0$$

Real task duration

$\delta_{it} \in \mathbb{N} \cup \{0\}$ gives the real duration of a task $t \in T$ and cost configuration $i \in I_t$.

> **Note**
>
> For leaf tasks, this real duration shall be directly defined as a constant for all cost configurations $i \in I_t$.

For non-leaf tasks, this real duration will be defined as:

$$\delta_{it} = \begin{cases} \sum_{c \in Ch_t} \delta_{cD_c[i]}, & \neg P_{NR}(t) \vee \neg P_{NA}(t) \\ 0, & P_{NR}(t) \wedge P_{NA}(t) \end{cases}$$

> **Quote**
>
> So for tasks with children, the real duration does not depend on the cost configuration. (only the cost configurations of leaf children)

Start/end constraints

$s_t \in \Theta(u_t) \cup \{\emptyset\}$ is the start (or null) of task $t \in T$.

$d_t \in \Theta(u_t) \cup \{\emptyset\}$ is the deadline (or null) of task $t \in T$.

$$P(t) := \exists i \in I_t \left[Ch_{it} \neq \emptyset\right]$$

$P(t)$ is true if t can possibly be a parent task.

$$\neg\exists p_1 \in T \exists p_2 \in T([P(p_1) \wedge P(p_2)] \rightarrow [Ch_{ip_1} \cup Ch_{ip_2} \neq \emptyset])$$

Each child must not have more than one possible parent.

> **Note**
>
> This doesn't say anything about a child possibly becoming #Deactivating orphans|deactivated if its parent doesn't choose a cost configuration including the child.

$\theta_t : I_t \rightarrow \text{Set } \mathbb{N}$ for a $t \in T$

σ_t is the scaling factor of parent timescale to child timescale.

$$\sigma_t = \frac{u_{Pa_t}}{u_t}$$

$$P_t[Pa] = Pa_t \neq \emptyset \rightarrow [s \geq \sigma_t D_{Pa_t}[s] \wedge s < \sigma_t(D_{Pa_t}[s] + 1)]$$

$P_t[Pa]$ states that the task's scheduled timescale instance occurs within the parent's scheduled timescale instance.

$$P_t[s] = s_t \neq \emptyset \rightarrow s \geq s_t$$

$P_t[s]$ states that the task's scheduled timescale instance occurs at or after the starting time constraint.

$$P_t[d] = d_t \neq \emptyset \rightarrow d < d_t$$

$P_t[d]$ states that the task's scheduled timescale instance occurs before the ending time constraint.

$$\theta_t(i) = \{s \in \Theta(u_t) | P_t[Pa] \wedge P_t[s] \wedge P_t[d]\}$$

$\theta_t(i)$ represents the set of all valid task starting times as determined by the given cost configuration and task and parent starting time/deadlines.

Real task completion time

For a given task we can find its actual completion time by considering the end time of its latest scheduled child.

If it is a leaf task, the worst-case scenario is the end of the time slot it is scheduled at.

$\epsilon : T \to \mathbb{N}$

$$\epsilon(t) = \begin{cases} \max_{c \in Ch_t} \epsilon(c), & Ch_t \neq \emptyset \\ u_t(D_t[s] + 1), & Ch_t = \emptyset \end{cases}$$

The real task completion time is used for #Prerequisites and computing #Cost. This is useful because oftentimes, large projects can be projected to complete before the end of the time slot it is scheduled in, simply by looking at when the last child task is finished (ex. you have a project scheduled for this month, but you will reasonably finish by the middle of the month).

This will not be used to constrain valid task starting times for the task itself. (as that would be circular).

This can sometimes have unintuitive results (such as multiple tasks having the exact same "real" completion time), thus, it may be helpful to rename this into something like "most specific ending time" in the future. (ex. the narrowest completion time)

Prerequisites

P_t is the set of prerequisite tasks of $t \in T$. (this should not contain cycles)

$$\forall t \in T \forall D_t[s] \in \theta(t) \forall p \in P(t) [\epsilon(p) \leq u_t D_t[s]]$$

We ensure that all task prerequisites are fulfilled before starting a task.

Non-overflow constraint

We ensure that no timescale instance overflows its timescale unit time under the current state.

$L : U \times \Theta(u) \to \text{Set } T$

$$L(u, s) = \{x | x \in T \wedge u_x = u \wedge D_x[s] = s\}$$

Function L gives the list of tasks under a given timescale unit and starting time.

$$\forall u \in U \forall s \in \Theta(u) \left(\sum_{t \in L(u,s)} \delta_{tD_t[i]} \leq u \right)$$

Decision variables

$D_t[i] \in I_t$ is the cost configuration chosen for task $t \in T$.

$D_t[s] \in \theta(t)$ is the starting time chosen for the task.

Objective function

We want to minimize the total cost under the chosen cost configurations.

$$\min \sum_{t \in T} [\iota x (x \in C_{D_t[i]t} \wedge u_t D_t[s] \in x_I)]_c$$

> **Note**
>
> This definition means that the cost of parents is weighted equally to the cost of children. Unlike duration, there is no the "real cost" for non-leaf tasks is choice between 0 and multiple constant values according to the available cost configs (according to #Deactivating orphans).

Deadlines

Sometimes a task's logical start and end time constraints exceed the actual "deadline" involved for the task.

1. Suppose a project P (timescale week) is due on Wednesday of week 2.
2. I am planning to begin work on Monday of week 1.
3. P has 3 subtasks P_1, P_2, P_3 (timescale day).
4. I can only do P_1 and P_2 on week 1, P_3 must be scheduled on week 2.
5. Semantically speaking, I should be able to schedule P_3 on week 2.
6. However, this requires me to set the logical deadline of P on week 2,

which technically implies time could be scheduled **after** Wednesday of week 2.
7. The solution is simply to specify the appropriate (more specific) logical deadline for the subtasks P_1, P_2, P_3.
8. In essence, the logical deadline of a task of a time unit u and actual deadline d is $\left\lceil \frac{d}{u} \right\rceil$

Default cost

It is not necessarily trivial to figure out cost.

We use a default cost for all tasks (good enough, usually user cares more about logical constraints and can assume all tasks have equal cost) e.g. 1000 for every task.

Quantizing Events

Often we want to factor in "events" into a schedule, portions of time which are blocked off for certain purposes.

Let's suppose an event's unit is τ s.t. $N\tau = \Upsilon$. That is, the atomic unit is greater than the "real time" unit.

If the opposite is true, it becomes trivial to convert an event's units into atomic units.

Let the set of all events be V.

A particular event is a tuple $(s, e) \in V$ s.t. $e > s$ and $e \pmod \tau \equiv 0$ and $s \pmod \tau \equiv 0$.

A process exists called "quantization" that can convert a set of non-overlapping events V into a set of time allocations A of unit u s.t. no timescale overflows, the relative order of events is reflected in the task, the difference between the starting time of the task and event $< u$ and the difference between the total task duration and the event duration $< \Upsilon$.

The set of all time allocations is a set A. A time allocation is a value (t, d) where $ut \pmod \Upsilon \equiv 0$ and $d \pmod \Upsilon \equiv 0$. t representing the timescale instance the time allocation pertains to and d representing the amount of time (in terms of the atomic unit) is allocated.

> **Note**
>
> A time allocation is a non-movable task. It is essentially the equivalent of a task that must start and end at exactly one timescale instance. Indeed, we can convert the set A into values $\in T$.

$$Q : V \to A^*$$

Where:

$$\forall a \in A \left[u_t = u \right]$$

#Non-overflow constraint

$$\forall a \in V \forall b \in V \left[a \neq b \land a_s \leq b_s \to Q(a)_t \leq Q(b)_t \right]$$

$$\forall v \in V \left[N v_s - u Q(v)_t < u \right]$$

$$\forall v \in V \left[Q(v)_d - (v_e - v_s) < \Upsilon \right]$$

External factors

One awkward part of the scheduling with only the above model is "finding deadlines" and "finding costs" when they don't exist.

Certain tasks are self-defined and do not necessarily come with the notion of a deadline, they simply are scheduled "when you're free".

However, there still exist certain schedules that are superior over others under uncertainty due to factors relating to human psychology and external considerations, these are what we will call "external factors".

Early biasing

We want to be able to force the scheduler to avoid regions of "empty space" in the middle of the schedule which may arise if there are not enough defined tasks to saturate the entire scheduling horizon defined.

Furthermore, certain external factors can influence task "urgency" (ex. responsibility, profitability), it is reasonable that *ceteris paribus*, one should be able to bias more "externally important" tasks towards earlier completion and have a schedule that incurs less cost overall from changes in deadlines or durations.

Thus, The cost $f_E(t)$ associated for any task t scales linearly with the scheduled time it is in and is parametrized by two constants. Namely:

$$f_E(t) = P_t K_E D_t[s] u_t$$

Where:

- K_E is a constant that scales the size of the early bias cost across all tasks.
- P_t is a constant that scales the size of early bias cost for this particular task.

The complete cost of all tasks due to early biasing is simply the sum of all individual task costs.

$$F_E = \sum_{t \in T} f_E(t)$$

Cognitive performance

Humans' cognitive performance (attention, inhibition, and memory, measured non-subjectively) varies throughout the day, and is highly correlated with circadian rhythm [1] [2]. This means that cognitive performance will "peak" and "dip" in rhythmic fashion throughout the day. The exact times of these "peaks" and "troughs" varies between individuals [3], however, there usually occurs two in a 24-hour period, and they alternate with each other [1:1]. It is postulated that one peak occurs early after waking because of lowered SP (sleep propensity) [4] and the other close to bedtime because of the WMZ (wake maintenance zone) [5].

Task cognitive sensitivity

It follows that tasks more sensitive to attention, inhibition, and memory performance should be scheduled in times of peak cognitive performance. This will enable such tasks to be completed faster and more precisely. Conversely, tasks largely insensitive to cognitive performance should be scheduled outside of peak performing times, as to not waste this valuable time in the day.

Tasks that are usually more sensitive to cognitive performance include the following categories:

- Tasks involving decision-making or judgment.

- Tasks requiring complex chains of reasoning and large amounts of context.
- Unfamiliar or "learning" tasks.

Conversely, routine or "simple" tasks are largely insensitive to cognitive performance.

We shall model task cognitive sensitivity as a value $S_t \in [0, 1]$ where $[0, 1] \subseteq \mathbb{R}$. The cost is given by:

$$f_S(t) = K_S S_t$$

Where K_S is a constant scalar.

Similar to #Early%20biasing, the global cost is the sum across all tasks.

$$F_S = \sum_{t \in T} f_S(t)$$

Context-switching

A wealth of research supports the idea that humans have a limited capacity to perform multiple attention-demanding tasks concurrently [6].

Much of what appears to be "multitasking," particularly when tasks compete for central attention, involves rapid task switching or interference between concurrently active tasks [6:1] [7]. These switches often make responses slower and more error-prone immediately after a task switch [7:1]. Furthermore, switching from an unfinished task can leave a lingering "attention residue," which impairs performance on the subsequent task [8]. In one field study of information workers, interrupted work that was resumed later the same day was resumed after an average of 25 minutes and 26 seconds, with workers engaging in an average of 2.26 other work activities before returning to it [9].

Encoding context-switching cost

It is often appealing to multitask when attempting to avoid wasted time as a result of blockage (e.g. waiting on a response). However, this often leads to slower and less precise work overall, due to responses to #Context-switching, in contrast with the illusion of productivity it generates. The better alternative is to batch interruptions into a single block of time, while keeping larger contiguous chunks of work intact.

> **Note**
>
> This, of course, implies that you *need to keep track* of all the actual context switches that occurs throughout your work day. This includes all things that may induce context-switching, regardless of how important they are (ex. scrolling social media, messages & email, small "routine" tasks) to have a significant effect on reducing context-switching overhead.

The mitigation of context-switching can be done without any additional constructs through the judicious setup of long task times for contiguous tasks of the 4-hour timescale unit.

One adjustment to the model that this may provide justification for is the variability of individual block sizes for tasks that utilize blocking. This would allow individual task "work blocks" to expand or shrink flexibly, and may allow for less "empty space" in the schedule and the necessity for context switching as a result of #Early%20biasing.

---

1. Dijk, Derk-Jan, Jeanne F. Duffy, and Charles A. Czeisler. “Circadian and Sleep/Wake Dependent Aspects of Subjective Alertness and Cognitive Performance.” Journal of Sleep Research 1, no. 2 (1992): 112–17. https://doi.org/10.1111/j.1365-2869.1992.tb00021.x. ↩ ↩

2. Chauhan, Satyam, Martina Vanova, Umisha Tailor, et al. “Chronotype and Synchrony Effects in Human Cognitive Performance: A Systematic Review.” Chronobiology International 42, no. 4 (2025): 463–99. https://doi.org/10.1080/07420528.2025.2490495. ↩

3. Munnilari, Madhavi, Tulasiram Bommasamudram, Judy Easow, et al. “Diurnal Variation in Variables Related to Cognitive Performance: A Systematic Review.” Sleep & Breathing = Schlaf & Atmung 28, no. 1 (2024): 495–510. https://doi.org/10.1007/s11325-023-02895-0. ↩

4. Bes, Frederik, Marc Jobert, and Hartmut Schulz. “Modeling Napping, Post-Lunch Dip, and Other Variations in Human Sleep Propensity.” Sleep 32, no. 3 (2009): 392–98. https://doi.org/10.1093/sleep/32.3.392. ↩

5. Shekleton, Julia A., Shantha M. W. Rajaratnam, Joshua J. Gooley, Eliza Van Reen, Charles A. Czeisler, and Steven W. Lockley. “Improved Neurobehavioral Performance during the Wake Maintenance Zone.” Journal of Clinical Sleep Medicine : JCSM : Official Publication of the American Academy of Sleep Medicine 9, no. 4 (2013): 353–62. https://doi.org/10.5664/jcsm.2588. ↩

6. Pashler, Harold. “Dual-Task Interference in Simple Tasks: Data and Theory.” Psychological Bulletin (US) 116, no. 2 (1994): 220–44. https://doi.org/10.1037/0033-2909.116.2.220. ↩ ↩

7. Monsell, Stephen. “Task Switching.” Trends in Cognitive Sciences 7, no. 3 (2003): 134–40. https://doi.org/10.1016/s1364-6613(03)00028-7. ↩ ↩

8. Leroy, Sophie. “Why Is It so Hard to Do My Work? The Challenge of Attention Residue When Switching between Work Tasks.” Organizational Behavior and Human Decision Processes 109, no. 2 (2009): 168–81. https://doi.org/10.1016/j.obhdp.2009.04.002. ↩

9. Mark, Gloria, Victor M. Gonzalez, and Justin Harris. “No Task Left behind? Examining the Nature of Fragmented Work.” Proceedings of the SIGCHI Conference on Human Factors in Computing

Systems (New York, NY, USA), CHI '05, April 2, 2005, 321–30. https://doi.org/10.1145/1054972.1055017. ↩